WELT 0 · DOPPELSTUNDEN 4 UND 5 · LEVEL 0.4 + 0.5

# W0·4 – Die Zauberschule: Eigene Methoden

Datei: `AQ_W0_Akademie_Geruest.txt` (Klassen Zauber und ZauberII) · IDE-Level 0.4 und 0.5 · 🎯 Im Lab: **Level 0.4 und 0.5** · Teil 1 in DS 4, Teil 2 in DS 5

Im obersten Turm der Akademie liegt die **Zauberschule**. Hier lernst du keine fertigen Zauber auswendig – hier erfindest du eigene **Zauberformeln**. Jede Formel hat einen **Namen**, braucht **Zutaten** und hat eine **Wirkung**: Ein Heilzauber der Stufe 3 heilt 45 Leben. In Java heißt eine solche Formel **Methode**: Die Zutaten sind die **Parameter**, die Wirkung ist der **Rückgabewert**.

Warum so viel Aufwand? In **allen 22 Abituraufgaben 2022–2026** musste implementiert werden – und jedes Mal war es eine Methode mit vorgegebener Signatur: „Implementieren Sie die Methode …“. Wer Signaturen sicher übersetzt, Rückgabepfade im Blick hat und weiß, was mit Parametern passiert, holt diese BE zuverlässig.

## Teil 1 – Zauberformeln: Methoden bauen und aufrufen (DS 4)

### Infoteil 1: Die Anatomie einer Methode

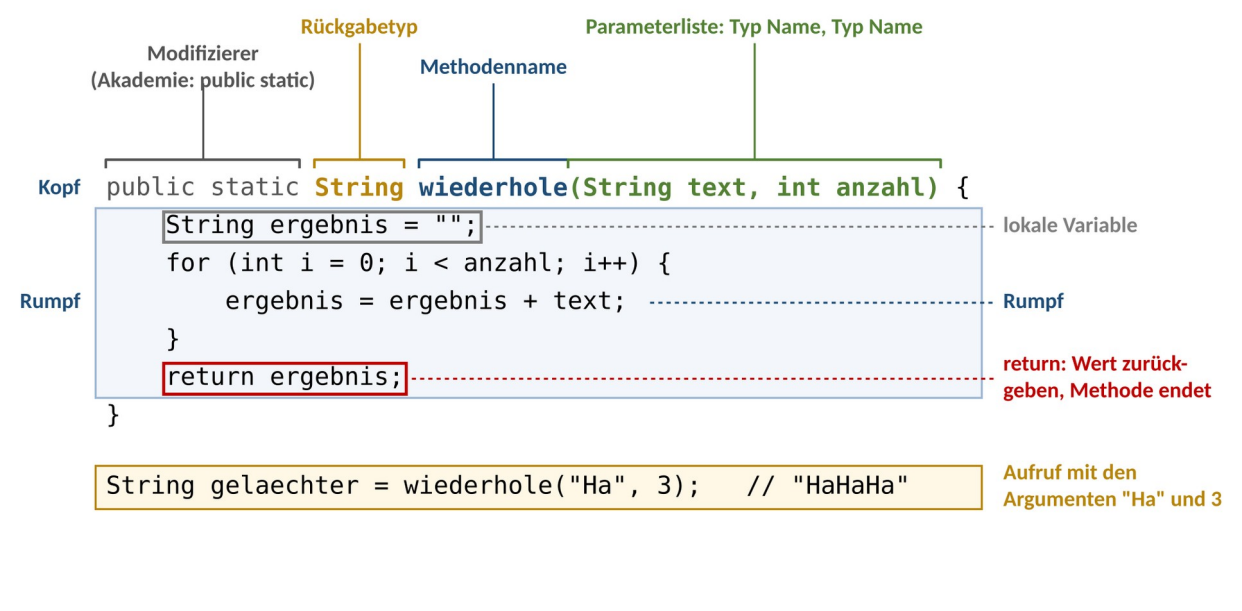



*Abbildung 1: Bestandteile der Methode wiederhole*

- Der **Kopf** legt fest, wie man die Methode benutzt: **Rückgabetyp** (was kommt heraus?), **Name**, **Parameterliste** (was muss man hineingeben?). Der **Rumpf** in geschweiften Klammern sagt, wie sie arbeitet.
- Die **Signatur** im Abitur ist der Kopf in sprachunabhängiger Form: `wiederhole(text: Zeichenkette, anzahl: Ganzzahl): Zeichenkette`.
- `static` bedeutet: Die Methode gehört zur Klasse selbst (eine **Klassenoperation**). In der Akademie sind alle Methoden `public static`; ab Welt 1 schreibst du Methoden für Objekte – dann ohne `static`.

## Infoteil 2: Von der Abi-Signatur zum Java-Methodenkopf

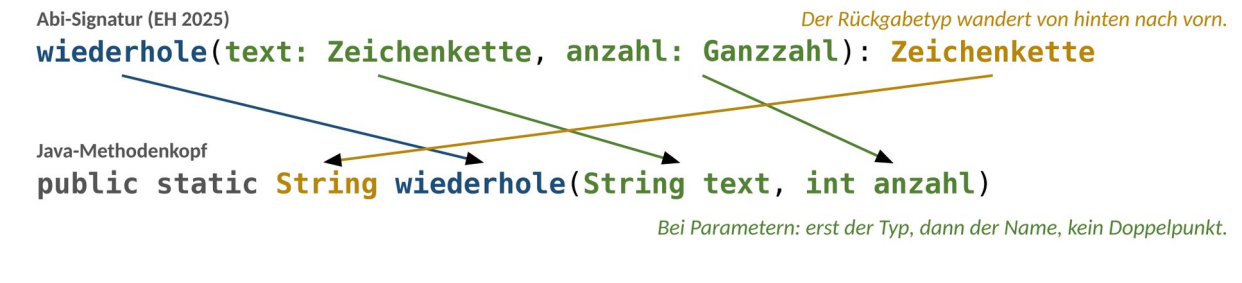



*Abbildung 2: Was wohin wandert*

| Abi-Signatur | Java-Methodenkopf |
|---|---|
| istLeer(r: Reihung von Ganzzahl): Wahrheitswert | `public static boolean istLeer(int[] r)` |
| mittel(a: Kommazahl, b: Kommazahl): Kommazahl | `public static double mittel(double a, double b)` |
| zeigeMenue(): nichts | `public static void zeigeMenue()` |
| initiale(name: Zeichenkette): Zeichen | `public static char initiale(String name)` |
| titelListe(n: Ganzzahl): Reihung von Zeichenkette | `public static String[] titelListe(int n)` |

1. Der **Rückgabetyp** wandert von hinten nach vorn; „nichts“ wird zu `void`.
2. Bei jedem **Parameter** steht erst der Typ, dann der Name – ohne Doppelpunkt. **Jeder** Parameter bekommt seinen eigenen Typ: `int a, int b` (nicht `int a, b`).
3. Die **Reihenfolge** der Parameter bleibt genau gleich. Keine Parameter: leere Klammern `()`.
4. Typnamen übersetzen: Ganzzahl → `int`, Kommazahl → `double`, Wahrheitswert → `boolean`, Zeichen → `char`, Zeichenkette → `String`, Reihung von X → `X[]` (vollständige Liste: AB 1, Teil B).

## Aufgabe 0: Blitz-Warm-up: Den Kopf zerlegen

PAPIER ★

Gegeben ist der Kopf `public static double trankPreis(int menge, boolean eilig, String sorte)`.

| Name | Rückgabetyp | Anzahl der Parameter | Typ des 2. Parameters | Signatur in Abi-Notation |
|---|---|---|---|---|
| | | | | |

## Aufgabe 1: Das Übersetzungsbüro der Zauberschule

PAPIER ★

**a)** Gib zu jeder Signatur den Java-Methodenkopf an (alle Methoden sind `public static`). **[3 BE]**

| Nr. | Abi-Signatur | Java-Methodenkopf |
|---|---|---|
| 1 | istMagisch(zahl: Ganzzahl): Wahrheitswert | |
| 2 | flaeche(breite: Kommazahl, hoehe: Kommazahl): Kommazahl | |
| 3 | begruesse(name: Zeichenkette): nichts | |
| 4 | letztesZeichen(s: Zeichenkette): Zeichen | |

| Nr. | Abi-Signatur | Java-Methodenkopf |
|---|---|---|
| 5 | zufallsReihung(laenge: Ganzzahl, max: Ganzzahl): Reihung von Ganzzahl | |
| 6 | startSignal(): nichts | |

**b)** Gib zu jedem Java-Methodenkopf die Signatur in Abi-Notation an. **[3 BE]**

| Nr. | Java-Methodenkopf | Abi-Signatur |
|---|---|---|
| 7 | `public static int anzahlZiffern(int n)` | |
| 8 | `public static String verbinde(String a, String b, char trenner)` | |
| 9 | `public static void leere(int[] regal)` | |
| 10 | `public static boolean enthaeltZeichen(String s, char c)` | |
| 11 | `public static double[] halbiert(int[] werte)` | |

## Aufgabe 2: Vom Auftrag zum Methodenkopf

PAPIER ★★

Im Abitur steht oft nur ein Text – die Signatur musst du dir selbst zusammensetzen. Gib zu jeder Beschreibung die Abi-Signatur **und** den Java-Methodenkopf an. Wähle sinnvolle Parameternamen.

| Nr. | Beschreibung | Abi-Signatur / Java-Methodenkopf |
|---|---|---|
| 1 | Die Methode **anzahlUnter** erhält eine Reihung von Ganzzahlen und eine Grenze und liefert, wie viele Werte kleiner als die Grenze sind. | |
| 2 | Die Methode **istVolljaehrig** prüft für ein Alter, ob es mindestens 18 ist. | |
| 3 | Die Methode **ausgeben** gibt alle Werte einer Reihung von Zeichenketten zeilenweise auf der Konsole aus. | |
| 4 | Die Methode **mittelwert** berechnet den Durchschnitt der Werte einer Reihung von Ganzzahlen. | |
| 5 | Die Methode **haeufigstesZeichen** liefert das Zeichen, das in einer Zeichenkette am häufigsten vorkommt. | |
| 6 | Die Methode **ohneDuplikate** liefert eine neue Reihung, die jeden Wert einer gegebenen Reihung von Ganzzahlen nur einmal enthält. | |

**Bewertung: [6 BE]**

### Infoteil 3: Aufrufen – Parameter und Argumente

Eine Methode wird erst durch einen **Aufruf** wirksam. Beim Aufruf übergibst du **Argumente**; sie werden der Reihe nach in die **Parameter** kopiert.

```
String a = wiederhole("Ha", 3);                  // Rückgabe speichern: "HaHaHa"
println(wiederhole("ab", 2) + "!");              // ausgeben: abab!
if (wiederhole("x", n).length() > 5) { … }       // in einer Bedingung verwenden
String b = wiederhole(wiederhole("x", 2), 3);    // als Argument: "xxxxxx"
wiederhole("Ha", 3);                             // erlaubt – aber die Rückgabe geht verloren!
```

| Begriff | Wo? | Beispiel |
|---|---|---|
| **Parameter** | im Methodenkopf: Variable mit Typ, ein Platzhalter | `String text`, `int anzahl` |
| **Argument** | beim Aufruf: konkreter Wert oder Ausdruck | `"Ha"`, `3`, `n + 1`, `wiederhole("x", 2)` |

- Anzahl, Reihenfolge und Typen der Argumente müssen zu den Parametern passen. Eine Ganzzahl darf an einen Kommazahl-Parameter übergeben werden (wird umgewandelt), eine Kommazahl an einen Ganzzahl-Parameter **nicht**.
- Der **Rückgabetyp** bestimmt, wo der Aufruf stehen darf: `double d = heilwert(4);` geht (Ganzzahl → Kommazahl), `int x = spruchtext("Eis", 1);` nicht.
- Aufruf **aus derselben Klasse**: nur der Name, z. B. `max2(a, b)`. Aufruf **aus dem Hauptprogramm** oder einer anderen Klasse: mit Klassennamen, z. B. `Zauber.heilwert(3)`.

## Aufgabe 3: Passt der Zauber?

PAPIER ★★

Die Klasse Zauber enthält (wie im Gerüst) diese drei Methoden:

```
public static int heilwert(int stufe)                    // Stufe * 15
public static boolean istStaerker(int a, int b)          // a > b
public static String spruchtext(String name, int stufe)  // z. B. "Feuerball (Stufe 3)"
```

**a)** Entscheide für jede Anweisung im Hauptprogramm, ob der Compiler sie akzeptiert. Gib bei akzeptierten Anweisungen den Wert bzw. die Ausgabe an, sonst den Grund. **[4 BE]**

| | Anweisung | ok? | Wert / Ausgabe bzw. Grund |
|---|---|---|---|
| A | `int h = Zauber.heilwert(2) + Zauber.heilwert(1);` | | |
| B | `String t = Zauber.spruchtext(3, "Blitz");` | | |
| C | `boolean b = Zauber.istStaerker(Zauber.heilwert(2), 25);` | | |
| D | `int x = Zauber.spruchtext("Eis", 1);` | | |
| E | `println(Zauber.spruchtext("Licht", Zauber.heilwert(1)));` | | |
| F | `double d = Zauber.heilwert(4);` | | |
| G | `int y = Zauber.heilwert(2.5);` | | |
| H | `Zauber.istStaerker(3);` | | |

**b)** Benenne für Anweisung C die Parameter und die Argumente beider Aufrufe. **[2 BE]**

### Infoteil 4: return – Rückgabepfade und void

- `return wert;` liefert den Wert an die Aufrufstelle und **beendet die Methode sofort**. Anweisungen direkt dahinter erreicht Java nie (Compilerfehler „unreachable statement“).
- Hat die Methode einen Rückgabetyp, muss **jeder mögliche Ablauf** mit einem `return` enden. Der Compiler prüft das, ohne die Bedingungen zu verstehen – sonst meldet er „missing return statement“.
- Eine `void`-Methode liefert **nichts**. Sie bewirkt etwas (Ausgabe, Änderung einer Reihung). Mit `return;` (ohne Wert) darf sie vorzeitig enden.
- Im Struktogramm: „gib … zurück“ – auch dort beendet es die Operation.

**Akzeptiert:** Das letzte `return` fängt alle übrigen Fälle.

```
public static int vorzeichen(int n) {
    if (n > 0) {
        return 1;
    } else if (n < 0) {
        return -1;
    }
    return 0;
}
```

**Abgelehnt** („missing return statement“) – obwohl inhaltlich jeder Fall abgedeckt ist:

```
public static int vorzeichen(int n) {
    if (n > 0) {
        return 1;
    } else if (n < 0) {
        return -1;
    } else if (n == 0) {
        return 0;
    }
}
```

## Aufgabe 4: Der Compiler-Detektiv

PAPIER ★★

Fünf Lehrlinge haben Zauberformeln abgegeben.

```
// M1
public static int klassifiziere(int n) {
    if (n > 0) { return 1; }
    if (n < 0) { return -1; }
    return 0;
}
// M2
public static String bewerte(int p) {
    if (p >= 10) { return "gut"; } else if (p >= 5) { return "ok"; }
}
// M3
public static int ersteGerade(int[] r) {
    for (int x : r) {
        if (x % 2 == 0) { return x; }
    }
    return -1;
}
// M4
public static boolean pruefe(int n) {
    return n > 5;
    println("geprüft");
}
// M5
public static void melde(int n) {
    if (n < 0) { return; }
    println("Wert: " + n);
}
```

**a)** Entscheide, welche Methoden der Compiler akzeptiert. Gib bei abgelehnten Methoden den Grund an, bei akzeptierten das Ergebnis des Aufrufs. **[5 BE]**

| | akzeptiert? | Grund bzw. Ergebnis für den Aufruf |
|---|---|---|
| M1 | | `klassifiziere(-7)`: |
| M2 | | |
| M3 | | `ersteGerade({3, 7, 4, 8})`: |
| M4 | | |
| M5 | | `melde(-3)`: |

**b)** Stelle M1 als Struktogramm nach EH § 5 dar. **[2 BE]**



**c)** Korrigiere M2 so, dass sie für alle Punktzahlen unter 5 "schwach" liefert. **[1 BE]**

### Infoteil 5: Lokale Variablen und Gültigkeit

- Variablen, die in einer Methode deklariert werden, und die **Parameter** sind **lokal**: Sie entstehen beim Aufruf und verschwinden beim Ende der Methode. Von außen sieht man sie nicht.
- Ein Parameter ist eine lokale Variable, die beim Aufruf mit dem **Wert des Arguments** vorbelegt wird.
- Gleicher Name heißt **nicht** gleiche Variable: Ein `kraft` im Hauptprogramm und ein Parameter `kraft` sind zwei verschiedene Kästchen.
- Eine Variable gilt nur in dem **Block** `{ … }`, in dem sie deklariert ist – die Zählvariable `i` einer for-Schleife also nur in der Schleife.

## Aufgabe 5: Wer sieht wen?

PAPIER ★★

```
1  public static int zauber(int kraft) {
2      int bonus = 5;
3      kraft = kraft + bonus;
4      return kraft * 2;
5  }
6  // Hauptprogramm
7  int kraft = 10;
8  int bonus = 1;
9  int ergebnis = zauber(kraft);
10 println(kraft + " " + bonus + " " + ergebnis);
```

**a)** Gib die Ausgabe an und begründe, warum sich kraft und bonus im Hauptprogramm nicht ändern. **[3 BE]**

Ein Lehrling hat diese Methode geschrieben und im Hauptprogramm `println(summe);` ergänzt:

```
1  public static int summeBis(int n) {
2      int summe = 0;
3      for (int i = 1; i <= n; i++) {
4          int quadrat = i * i;
5          summe = summe + quadrat;
6      }
7      println(i);
8      println(quadrat);
9      return summe;
10 }
```

**b)** Der Compiler meldet drei Fehler „cannot find symbol“. Gib die drei Stellen an (zwei in der Methode, eine im Hauptprogramm) und erläutere jeweils. **[3 BE]**

### Infoteil 6: Methode ruft Methode – das Aufrufprotokoll

Methoden dürfen andere Methoden aufrufen. Die aufrufende Methode **wartet**, bis die aufgerufene ihr Ergebnis liefert, und rechnet dann weiter. So zerlegt man Probleme in kleine, einzeln prüfbare Teile.

```
public static int quadrat(int x) {
    return x * x;
}

public static int summeDerQuadrate(int a, int b) {
    int summe = 0;   // → Tabellenzeile (Startwerte)
    for (int i = a; i <= b; i++) {
        summe = summe + quadrat(i);   // → Tabellenzeile
    }
    return summe;
}
```

| summeDerQuadrate(a: Ganzzahl, b: Ganzzahl): Ganzzahl |
|---|
| summe ← 0 |
| für i von a bis b |
| summe ← summe + quadrat(i) *// Aufruf der Hilfsmethode* |
| gib summe zurück |

**Aufrufprotokoll** für `summeDerQuadrate(2, 4)` – eine Zeile an jeder mit „// → Tabellenzeile“ markierten Stelle (vor der Schleife und bei jedem Aufruf der Hilfsmethode):

| i | Aufruf | Rückgabe | summe |
|---|---|---|---|
| vor der Schleife | – | – | 0 |
| 2 | quadrat(2) | 4 | 4 |
| 3 | quadrat(3) | 9 | 13 |
| 4 | quadrat(4) | 16 | 29 |
| Ende | | | gib 29 zurück |

## Aufgabe 6: Die Runen der Zauberschule

PAPIER ★★

Auf den Türen der Zauberschule stehen Runen. Ihr Wert wird mit zwei Methoden berechnet:

```
1  public static int runenwert(char c) {
2      if (c == 'X') {
3          return 10;
4      }
5      if (c == 'V') {
6          return 5;
7      }
8      if (c == 'I') {
9          return 1;
10     }
11     return 0;
12 }
13
14 public static int runenSumme(String s) {
15     int summe = 0;   // → Tabellenzeile (Startwerte)
16     for (int i = 0; i < s.length(); i++) {
17         summe = summe + runenwert(s.charAt(i));   // → Tabellenzeile
18     }
19     return summe;
20 }
```

**a)** Erstelle ein Aufrufprotokoll für `runenSumme("XVIIa")`: eine Zeile an jeder mit „// → Tabellenzeile“ markierten Stelle (vor der Schleife und bei jedem Aufruf von runenwert). Gib den Rückgabewert an. **[3 BE]**

| i | Aufruf von runenwert | Rückgabe | summe |
|---|---|---|---|
| vor der Schleife | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |

Rückgabewert: _____

**b)** Gib an, wie oft `runenwert` bei der Anweisung `int z = runenSumme("XXX") + runenSumme("");` aufgerufen wird, und den Wert von z. **[2 BE]**

**c)** Gegeben ist `public static int verdopple(int x) { return x * 2; }`. Gib für `int e = verdopple(3) + verdopple(verdopple(1));` die **Reihenfolge** aller Aufrufe mit Argument und Rückgabe sowie den Wert von e an. **[2 BE]**

**d)** Nenne zwei Vorteile davon, den Wert einer einzelnen Rune in eine eigene Methode auszulagern. **[2 BE]**

> 🎯 **Im Lab üben: Level 0.4 „Die Zauberschule: Eigene Methoden“ – Punkte gibt es dort automatisch**
>
> Passend zu Teil 1 (Infoteile 1–6, Aufgaben 0–7): Methodenköpfe zerlegen und übersetzen, Aufrufe prüfen, Code-Puzzle – jede Antwort wird sofort geprüft. Danach geht es in der Online-IDE mit IDE-Level 0.4 weiter.

## Aufgabe 7: Level 0.4 – Die Zauberschule

PAPIER → IDE ★ – ★★★

In der Klasse **Zauber** sind nur die Köpfe der Stufe Basis vorgegeben. Für Erweiterung und Experte steht im Gerüst **nur die Abi-Signatur** – den Java-Kopf schreibst du selbst. Damit das Programm trotzdem kompiliert, sind die passenden Tests auskommentiert. Du schaltest sie frei, sobald deine Köpfe stehen.

**a)** **Erst Papier:** Schreibe zu jeder Signatur den Java-Methodenkopf. **[3 BE]**

| Stufe | Abi-Signatur (aus dem Gerüst) | Java-Methodenkopf |
|---|---|---|
| ★★ | manaKosten(stufe: Ganzzahl, verstaerkt: Wahrheitswert): Ganzzahl | |
| ★★ | kannZaubern(mana: Ganzzahl, stufe: Ganzzahl, verstaerkt: Wahrheitswert): Wahrheitswert | |
| ★★ | max3(a: Ganzzahl, b: Ganzzahl, c: Ganzzahl): Ganzzahl | |
| ★★★ | runden(wert: Kommazahl): Ganzzahl | |
| ★★★ | wuerfelImBereich(von: Ganzzahl, bis: Ganzzahl): Ganzzahl | |
| ★★★ | zauberkette(stufe: Ganzzahl): Zeichenkette | |

**b)** **Basis in der IDE:** Implementiere `heilwert`, `istStaerker`, `spruchtext` und die Hilfsmethode `max2` (Köpfe vorgegeben). Starte die Testklasse `L04ZauberTest`: Alle Tests mit `basis_` müssen grün sein.

**c)** **Erweiterung:** Tippe deine Köpfe aus a) ab und schreibe die Rümpfe. `kannZaubern` muss `manaKosten` **aufrufen**, `max3` muss `max2` **aufrufen**. Dann schaltest du frei – in **zwei** Blöcken (in `L04ZauberTest` und im Hauptprogramm). Lösche jeweils genau die beiden Markerzeilen:

```
/* LEVEL-0.4-START Erweiterung        <- diese ganze Zeile löschen

    @Test
    public void erweiterung_manaKosten() {
        ...
    }
    ...

    LEVEL-0.4-ENDE Erweiterung */         <- diese ganze Zeile löschen
```

**d) Experte:** genauso mit den Markern `LEVEL-0.4-START Experte` und `LEVEL-0.4-ENDE Experte`. Runden und Zufallszahl kennst du aus AB 2, Infoteil 3 – `Math.round` ist tabu.

### Checkliste zum Freischalten

- ☐ Alle drei Methoden der Stufe haben einen Kopf (Name, Parametertypen, Rückgabetyp wie in der Signatur).
- ☐ Beide Blöcke der Stufe freigeschaltet: in `L04ZauberTest` **und** im Hauptprogramm – je zwei Markerzeilen gelöscht, sonst nichts.
- ☐ Meldet der Compiler „Methode nicht gefunden“ o. Ä.: Kopf mit der Signatur vergleichen – **niemals** den Test ändern.
- ☐ Code-Review im Team: Ruft kannZaubern wirklich manaKosten auf, max3 wirklich max2? (Das kann kein Test prüfen.)

| Stufe | Methoden in Zauber | Tests beginnen mit |
|---|---|---|
| ★ Basis | heilwert, istStaerker, spruchtext, max2 (Köpfe vorgegeben) | `basis_` |
| ★★ Erweiterung | manaKosten, kannZaubern, max3 (Köpfe selbst schreiben, freischalten) | `erweiterung_` |
| ★★★ Experte | runden, wuerfelImBereich, zauberkette (Köpfe selbst schreiben, freischalten) | `experte_` |

### Für Schnelle (Teil 1): Zauberformeln, die Zauberformeln rufen

- **Echte römische Zahlen:** Erweitere die Runen aus Aufgabe 6 um L (50), C (100), D (500), M (1000) und um die Subtraktionsregel: Steht ein kleinerer Wert vor einem größeren, wird er abgezogen („XIV“ = 14, „MCMXCIV“ = 1994).
- **kgV:** `kgV(a: Ganzzahl, b: Ganzzahl): Ganzzahl` mit `Werkstatt.ggT` – es gilt kgV(a, b) = a / ggT(a, b) * b.
- **Kalender der Akademie:** `istSchaltjahr(jahr: Ganzzahl): Wahrheitswert` (durch 4 teilbar, aber nicht durch 100 – außer durch 400) und `tageImMonat(monat: Ganzzahl, jahr: Ganzzahl): Ganzzahl`, die istSchaltjahr aufruft.

Side-Quest: freiwillig und ohne Punkte – stellt eure Ergebnisse im Team oder der Lehrkraft vor.

# Teil 2 – Zaubern mit Reihungen: Wert und Referenz (DS 5)

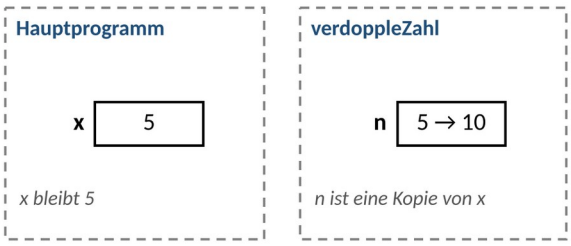
Bisher hatten deine Zauberformeln einfache Zutaten: Zahlen, Zeichen, Zeichenketten. Jetzt bekommen sie ganze **Reihungen** – und liefern Reihungen zurück. Dabei passiert etwas Überraschendes: Manche Änderungen in der Methode sind danach im Hauptprogramm sichtbar, andere nicht.

### Infoteil 7: Wertübergabe – aber was ist der Wert?

Java übergibt beim Aufruf immer eine **Kopie des Werts** der Argumentvariablen. Entscheidend ist, **was** in der Variablen steht:

## Fall 1: Ganzzahl-Parameter

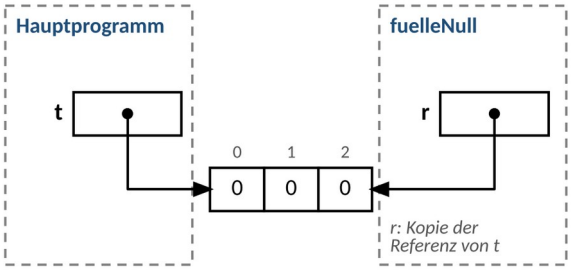
```
public static void verdoppleZahl(int n) {
    n = n * 2;
}
// Hauptprogramm
int x = 5;
verdoppleZahl(x);
println(x);           // 5
```




**Keine Wirkung nach außen:** n ist eine Kopie der Zahl 5. n wird 10, x bleibt 5.

## Fall 2: Reihungs-Parameter, Inhalt ändern

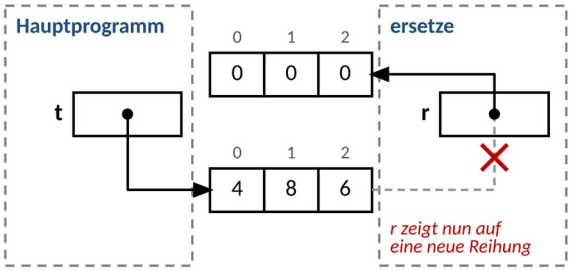
```
public static void fuelleNull(int[] r) {
    for (int i = 0; i < r.length; i++) {
        r[i] = 0;
    }
}
// Hauptprogramm
int[] t = {4, 8, 6};
fuelleNull(t);
println(t[0]);        // 0
```




**Wirkung nach außen:** r ist eine Kopie der **Referenz**. Beide Pfeile zeigen auf dieselbe Reihung – was über r geändert wird, sieht auch t.

## Fall 3: Reihungs-Parameter neu belegen

```
public static void ersetze(int[] r) {
    r = new int[r.length];
}
// Hauptprogramm
int[] t = {4, 8, 6};
ersetze(t);
println(t[0]);        // 4
```




**Keine Wirkung nach außen:** Nur der lokale Pfeil r wird auf eine neue Reihung umgelenkt. t zeigt weiter auf die alte.

### Merke: Wert und Referenz

- Einen Parameter neu zu belegen (`n = …`, `r = new …`) wirkt **nie** nach außen.
- Den **Inhalt** einer übergebenen Reihung zu ändern (`r[i] = …`) wirkt **immer** nach außen.
- Soll eine Methode „eine neue Reihung liefern“, muss sie mit `new` eine anlegen und zurückgeben – sonst verändert sie das Original.

## Aufgabe 8: Original oder Kopie?

PAPIER ★★

Vier Zauberformeln und ein Hauptprogramm. Überlege zuerst ohne Gerät!

```
1  public static void tausche(int a, int b) {
2      int h = a;
3      a = b;
4      b = h;
5  }
6  public static void tauscheInReihung(int[] r, int i, int j) {
7      int h = r[i];
8      r[i] = r[j];
9      r[j] = h;
10 }
11 public static int[] mitNull(int[] r) {
12     r[0] = 0;
13     return r;
14 }
15 public static void neuBelegen(int[] r) {
16     r = new int[3];
17     r[1] = 1;
18 }
19 // Hauptprogramm
20 int x = 1;
21 int y = 2;
22 int[] z = {3, 4, 5};
23 tausche(x, y);
24 tauscheInReihung(z, 0, 2);
25 int[] w = mitNull(z);
26 neuBelegen(w);
27 w[2] = 9;
28 println(x + " " + y);
29 println(z[0] + " " + z[1] + " " + z[2]);
30 println(w[0] + " " + w[1] + " " + w[2]);
31 println(z == w);
```

**a)** Gib die vier Ausgabezeilen an. **[4 BE]**

| Ausgabe 1 | Ausgabe 2 | Ausgabe 3 | Ausgabe 4 |
|---|---|---|---|
| | | | |

**b)** Zeichne das Speicherbild (1) direkt nach Zeile 25 und (2) während `neuBelegen` kurz vor dem Ende der Methode. **[3 BE]**

**c)** Begründe für jede der vier Methoden, ob ihre Änderung im Hauptprogramm sichtbar ist. **[4 BE]**

### Infoteil 8: In der Reihung arbeiten oder neue Reihung liefern?

Abituraufgaben sagen genau, was gewünscht ist. Achte auf die Formulierung – sie entscheidet über den Rückgabetyp:

| | in-place: die Reihung verändern | neue Reihung liefern |
|---|---|---|
| typische Formulierung | „… verändert r“, „setzt alle …“, „sortiert r“ | „… liefert eine neue Reihung …“, „r bleibt unverändert“ |
| Rückgabetyp | nichts (`void`) | Reihung von … (`int[]`) |
| Muster | `r[i] = …` in einer Zählschleife | `int[] neu = new int[…];` füllen, `return neu;` |
| Beispiel | `halbiereAlle({9, 4, 7})` → r wird {4, 2, 3} | `halbiert({9, 4, 7})` liefert {4, 2, 3}, r bleibt {9, 4, 7} |

```
public static void halbiereAlle(int[] r) {
    for (int i = 0; i < r.length; i++) {
        r[i] = r[i] / 2;
    }
}
```

```
public static int[] halbiert(int[] r) {
    int[] neu = new int[r.length];
    for (int i = 0; i < r.length; i++) {
        neu[i] = r[i] / 2;
    }
    return neu;
}
```

## Aufgabe 9: In der Reihung oder neu?

PAPIER ★★

**a)** Gib zu jeder Beschreibung die Abi-Signatur an und kreuze an, ob die Methode in-place arbeitet oder eine neue Reihung liefert. **[4 BE]**

| Nr. | Beschreibung | Abi-Signatur | in-place / neu |
|---|---|---|---|
| 1 | **sortiere** ordnet die Werte der Reihung r aufsteigend. | | ☐ in-place ☐ neu |
| 2 | **gespiegelt** liefert eine Reihung mit den Werten von r in umgekehrter Reihenfolge; r bleibt unverändert. | | ☐ in-place ☐ neu |
| 3 | **nullStattNegativ** setzt alle negativen Werte in r auf 0. | | ☐ in-place ☐ neu |
| 4 | **positionenVon** liefert alle Indizes, an denen x in r vorkommt. | | ☐ in-place ☐ neu |

**b)** Implementiere `nullStattNegativ` aus a) und die Methode `begrenzt(r: Reihung von Ganzzahl, max: Ganzzahl): Reihung von Ganzzahl`. Sie liefert eine neue Reihung, in der jeder Wert, der größer als max ist, durch max ersetzt ist; r bleibt unverändert. **[4 BE]**

### Infoteil 9: Hilfsmethoden – große Zauber aus kleinen

Eine **Hilfsmethode** löst ein Teilproblem, das sonst die Hauptmethode unübersichtlich machen würde. Im Abitur steht oft dabei: „Die Methode … darf verwendet werden.“ – dann **sollst** du sie aufrufen.

```
public static boolean istVokal(char c) {
    return c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u'
        || c == 'A' || c == 'E' || c == 'I' || c == 'O' || c == 'U';
}

public static int anzahlVokale(String s) {
    int anzahl = 0;
    for (int i = 0; i < s.length(); i++) {
        if (istVokal(s.charAt(i))) {      // Teilproblem ausgelagert
            anzahl++;
        }
    }
    return anzahl;
}
```

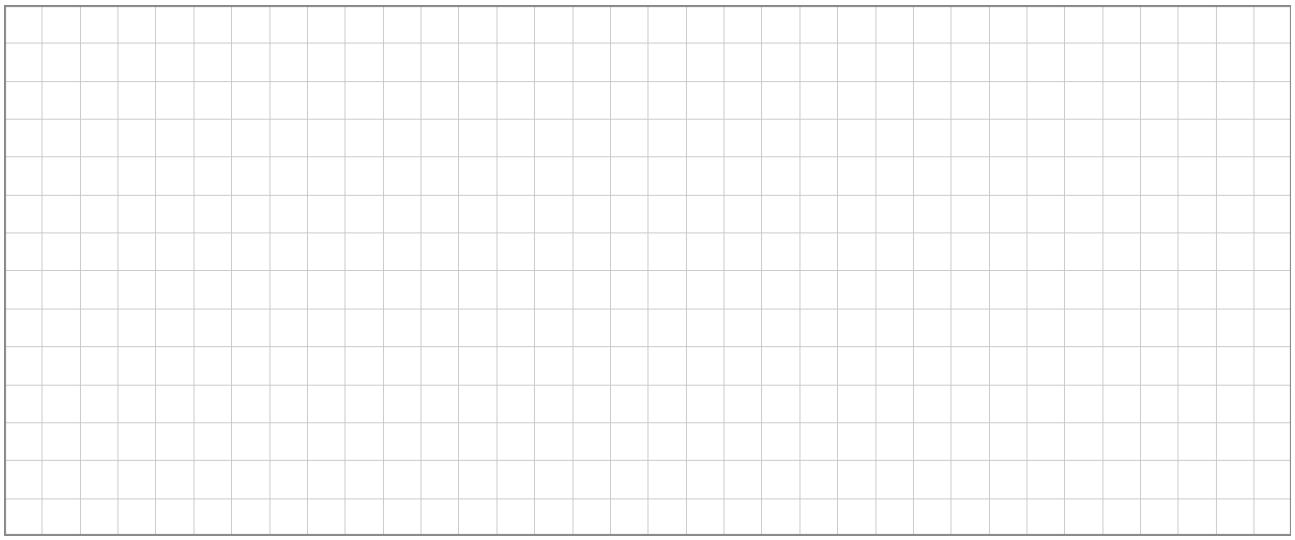
- **Zerlegen:** Welches Teilproblem kommt mehrfach vor oder lässt sich in einem Satz beschreiben? → eigene Methode mit eigener Signatur.
- **Vorteile:** kürzerer, lesbarer Code; jede Methode einzeln testbar; Fehler nur an einer Stelle zu korrigieren; Wiederverwendung.
- **Standardbausteine als Methoden:** Summe, Zählen, Maximum mit Index, lineare Suche, neue Reihung füllen – in Welt 2 werden sie zum Werkzeugkasten.

## Aufgabe 10: Das vokalreichste Zauberwort

PAPIER ★★★

**Auftrag:** Die Methode `bestesWort(woerter: Reihung von Zeichenkette): Zeichenkette` liefert das Wort mit den meisten Vokalen. Bei Gleichstand wird das erste dieser Wörter geliefert, für eine leere Reihung die leere Zeichenkette "". Die Methode `anzahlVokale` aus Infoteil 9 darf verwendet werden.

**a)** Implementiere `bestesWort` in Java. **[4 BE]**



**b)** Gib an, was `bestesWort` für {"Eis", "Feuer", "Erde"} und für {"Wind", "Blitz"} liefert. **[1 BE]**

**c)** Gib an, wie oft `istVokal` beim Aufruf `anzahlVokale("Feuer")` aufgerufen wird, und begründe. **[1 BE]**

**🎯 Im Lab üben: Level 0.5 „Zauberschule II: Wert und Referenz“ – Punkte gibt es dort automatisch**

Passend zu Teil 2 (Infoteile 7–9, Aufgaben 8–12): Wert oder Referenz, Original oder Kopie, Lückencode und Bug-Hunt – jede Antwort wird sofort geprüft. Danach geht es in der Online-IDE mit IDE-Level 0.5 weiter.

## Aufgabe 11: Level 0.5 – Die Zauberschule, Teil II

IDE ★ – ★★★

**a)** Arbeite in der Klasse **ZauberII** (Köpfe vorgegeben), Testklasse `L05ZauberIITest`.

| Stufe | Methoden (Signatur wie im Abitur) | Tests beginnen mit |
|---|---|---|
| ★ Basis | fuelle(r: Reihung von Ganzzahl, wert: Ganzzahl): nichts<br>erhoeheAlle(r: Reihung von Ganzzahl, bonus: Ganzzahl): nichts<br>erhoehteKopie(r: Reihung von Ganzzahl, bonus: Ganzzahl): Reihung von Ganzzahl | `basis_` |
| ★★ Erweiterung | istImBereich(x: Ganzzahl, von: Ganzzahl, bis: Ganzzahl): Wahrheitswert<br>zaehleImBereich(r: Reihung von Ganzzahl, von: Ganzzahl, bis: Ganzzahl): Ganzzahl<br>indexDesMinimums(r: Reihung von Ganzzahl): Ganzzahl<br>tauscheErstesUndLetztes(r: Reihung von Ganzzahl): nichts | `erweiterung_` |
| ★★★ Experte | zweitgroesster(r: Reihung von Ganzzahl): Ganzzahl<br>verschmelze(a: Reihung von Ganzzahl, b: Reihung von Ganzzahl): Reihung von Ganzzahl | `experte_` |

- Bei `erhoehteKopie` prüfen die Tests **beides**: das Ergebnis **und** dass r unverändert ist.
- `zaehleImBereich` soll die Hilfsmethode `istImBereich` aufrufen (Code-Review im Team).
- `verschmelze` arbeitet nach dem Reißverschluss-Prinzip: immer das kleinere der beiden vorderen Elemente nehmen, am Ende den Rest anhängen. Das ist der Kern von Mergesort (Welt 3).

**b)** **Vorhersage ohne Gerät:** Das Hauptprogramm ruft mit `staerken = {12, 3, 25, 3, 18}` nacheinander `zaehleImBereich(staerken, 10, 20)`, `indexDesMinimums(staerken)` und `zweitgroesster(staerken)` auf. Notiere die drei Ausgaben, starte dann und vergleiche.

---

## Aufgabe 12: Bug-Hunt in der Zauberschule

PAPIER ★★

In jeder Methode steckt **genau ein** Fehler.

```
// A: soll die größere der beiden Zahlen liefern
public static void max(int a, int b) {
    if (a > b) {
        return a;
    }
    return b;
}
// B: soll eine neue Reihung ohne das erste Element liefern (r ist nicht leer)
public static int[] ohneErstes(int[] r) {
    int[] neu = new int[r.length - 1];
    for (int i = 0; i < neu.length; i++) {
        neu[i] = r[i];
    }
    return neu;
}
// C: soll zählen, wie viele Werte größer als grenze sind
public static int zaehleUeber(int[] r, int grenze) {
    int anzahl = 0;
    grenze = 0;
    for (int i = 0; i < r.length; i++) {
        if (r[i] > grenze) {
            anzahl++;
        }
    }
    return anzahl;
}
// D: soll prüfen, ob das Gold für anzahl Tränke zum Preis preis reicht
public static int gesamtpreis(int preis, int anzahl) {
    return preis * anzahl;
}
public static boolean kannKaufen(int gold, int preis, int anzahl) {
    return gesamtpreis(anzahl, gold) <= gold;
}
```

**a)** Beschreibe für jede Methode den Fehler und gib die korrigierte Zeile an. **[4 BE]**

| Methode | Fehler | korrigierte Zeile |
|---|---|---|
| A | | |
| B | | |
| C | | |
| D | | |

**b)** Gib für B, C und D je einen Aufruf mit dem falschen und dem richtigen Ergebnis an. Welcher Fehler wird schon vom Compiler gemeldet? **[3 BE]**

### Für Schnelle (Teil 2): Meisterzauber

- **Rotation:** `rotiereLinks(r: Reihung von Ganzzahl, k: Ganzzahl): nichts` schiebt alle Werte k Stellen nach links (vorne Herausfallende kommen hinten wieder herein): {1, 2, 3, 4, 5} mit k = 2 wird {3, 4, 5, 1, 2}. Zerlege: Schreibe zuerst `rotiereEinmal(r)`.
- **Anagramm:** `istAnagramm(s: Zeichenkette, t: Zeichenkette): Wahrheitswert` – gleiche Buchstaben in anderer Reihenfolge („lager“ und „regal“). Nutze `Vorrat.zaehleZeichen` aus Level 0.3.
- **Spur des Reißverschlusses:** Verfolge `verschmelze({1, 4, 9}, {2, 3, 10})` in einer Tracetabelle mit den Spalten i, j, k und neu. Wann endet die erste Schleife – und wer liefert den Rest?

Side-Quest: freiwillig und ohne Punkte – stellt eure Ergebnisse im Team oder der Lehrkraft vor.

### Abi-Check für Methoden – vor jeder Abgabe

- ☐ Kopf exakt nach Signatur: Name, Reihenfolge und Typen der Parameter, Rückgabetyp („nichts“ = `void`).
- ☐ Jeder mögliche Ablauf endet mit einem `return` passenden Typs; nichts steht hinter einem `return`.
- ☐ Parameter werden benutzt, nicht mit festen Werten überschrieben.
- ☐ „neue Reihung“ verlangt? → `new` und das Original nur lesen. „verändert r“ verlangt? → `r[i] = …`, kein `new`.
- ☐ Darf eine Hilfsmethode verwendet werden, rufst du sie auf – mit Argumenten in der richtigen Reihenfolge.